

Universidad Mariano Gálvez de Guatemala
Ingeniería En Sistemas de Información
Sede Chiquimulilla

Curso: ASEGURAMIENTO DE LA CALIDAD DE SOFTWARE
Docente: CARMELO ESTUARDO MAYÉN MONTERROSO



Mapa conceptual e investigación

Estudiantes:

Sauly Estefania Cermeño Morales

Carnet:

179-21-8130

Chiquimulilla Santa Rosa 18 de julio de 2025

Aserciones en Chai.

Chai es una biblioteca de aserciones muy popular en el ecosistema de JavaScript, conocida por su versatilidad y compatibilidad con frameworks de pruebas como Mocha, Jasmine y Cypress. Las aserciones son sentencias que verifican si una condición es verdadera; si no lo es, la aserción falla, señalando un error en el código. Similar al módulo ``assert`` nativo de Node.js, Chai facilita las pruebas al proporcionar una amplia gama de aserciones con una sintaxis clara, expresiva y fácil de usar, lo que mejora la legibilidad y agiliza el proceso de desarrollo.

Requisitos para el uso de Chai

Tener un conocimiento básico de JavaScript y conceptos de prueba. Además, asegúrese de que Node.js está instalado en su entorno de desarrollo, ya que es necesario para ejecutar pruebas Chai.

Instalación de Chai

Comience abriendo su terminal en el directorio raíz de su proyecto. Luego, ejecute el siguiente comando para instalar Chai como dependencia de desarrollo:

(instalación de npm - save-dev chai)

Este comando agrega Chai a su proyecto, lo que le permite utilizar sus capacidades de afirmación en sus pruebas

Explorando los estilos de afirmación de Chai

Chai brinda a los desarrolladores la flexibilidad de elegir entre tres estilos de aserción distintos: Debería, Esperar y Afirmar. Cada estilo ofrece una sintaxis única pero mantiene una funcionalidad subyacente consistente. Comprender estos estilos le ayudará a escribir pruebas que no solo sean efectivas sino que también se alineen con sus preferencias de codificación.

El estilo Should extiende cada objeto con una propiedad ``should``, lo que permite un lenguaje legible y encadenable para afirmaciones. Es particularmente popular por su

gran parecido con el lenguaje natural, lo que hace que las pruebas sean fáciles de leer y escribir:

```
( dejar debería = chai.debería();  
(4).debería.be.arriba(3); // Afirma que 4 es mayor que 3  
"nombre".debería.be.a('cadena'); // Afirma que "nombre" es una cadena  
debería.existir(nulo); // Demuestra que null no existe )
```

Expect ofrece la misma legibilidad que Should, con una sintaxis ligeramente diferente y sin los posibles peligros de ampliar el prototipo de Object.

Referencia de API

La API Chai se desglosa por estilo o tarea.

Estilos de afirmación

El Esperar / debería API cubre los estilos de afirmación BDD.

El Afirmary API cubre el estilo de afirmación TDD.

Complementos

El API de complemento será de utilidad para cualquier persona interesada en construir complementos como ayudantes para secar sus pruebas o para liberar a la comunidad.

Herramientas

El Suite de prueba en línea es una carrera en vivo de la suite de prueba de Chai. Úselo para garantizar la compatibilidad del navegador.

Chai es infinitamente más poderoso de lo que se incluye, limitado solo por lo que desea lograr. La API del complemento También se pretende como una forma de simplificar las pruebas al proporcionar a los usuarios una forma de Encapsular afirmaciones comunes para el uso de repetición.

Exponer globales en complementos

Al crear un complemento Chai, es posible exponer globales que se pueden usar en múltiples archivos. Aquí le mostramos cómo hacerlo de manera sostenible:

Buena práctica

Prefiere exportar cualquier global en el registro del módulo para que pueda importarse directamente en lugar de agregarlo como una propiedad en el objeto Chai:

```
// An example of a good plugin:  
  
export const myGlobal = {...};  
  
export default function myPlugin(chai, utils) {  
}
```

Problemas potenciales

Evite exponer a los globales solo a través de `chai.use()` sin hacerlos disponibles para importar, ya que esto puede conducir a problemas al intentar usar el global en múltiples archivos:

```
// An example of a plugin which may have issues:  
  
const myGlobal = {...};  
  
export default function myPlugin(chai, utils) {  
  chai.myGlobal = myGlobal;  
}  
  
// Another example of a plugin which may have issues:  
  
export default function myPlugin(chai, utils) {  
  chai.myGlobal = {...};  
}
```

Proteger contra múltiples llamadas a use(..)

En ciertas situaciones el use(..) La función podría llamarse varias veces con su complemento. Para muchos complementos, esto no será un problema, pero se considera una mejor práctica verificar si el complemento ya se ha aplicado. Aquí hay un ejemplo artificial de cómo puede implementar un cheque en su complemento, pero la implementación real se deja en el autor del complemento.

```
import * as chai from 'chai';

let overwritten = false;

function somePlugin(base) {
  if (!overwritten) {
    base.util.overwriteMethod(base.Assertion.prototype, "equal", function (_super)
    {
      return function(...args) {
        console.log("Called!"); // log something out

        return _super.call(this, ...args);
      };
    });
    overwritten = true;
  }
}

chai.use(somePlugin);
chai.use(somePlugin);

chai.expect(123).to.equal(123); // Logs `Called!` only once
```

Chai npm

Node.js

chai está disponible en **mpn**. Para instalarlo, escriba:

```
$ npm install --save-dev chai
```

Navegadores

También puede usarlo dentro del navegador; instalar a través de NPM y usar el chai.js Archivo encontrado dentro de la descarga. Por ejemplo:

```
<script src="./node_modules/chai/chai.js"></script>
```

Uso

Importe la biblioteca en su código y luego elija uno de los estilos que le gustaría usar - assert, expect o should:

```
import { assert } from 'chai'; // Using Assert style
```

```
import { expect } from 'chai'; // Using Expect style
```

```
import { should } from 'chai'; // Using Should style
```

Registre el estilo de prueba Chai a nivel mundial

```
import 'chai/register-assert'; // Using Assert style
```

```
import 'chai/register-expect'; // Using Expect style
```

```
import 'chai/register-should'; // Using Should style
```

Estilos de afirmación de importación como variables locales

```
import { assert } from 'chai'; // Using Assert style
```

```
import { expect } from 'chai'; // Using Expect style
```

```
import { should } from 'chai'; // Using Should style
```

```
should(); // Modifies `Object.prototype`
```

```
import { expect, use } from 'chai'; // Creates local variables `expect` and `use`;  
useful for plugin use
```

Uso con mocha

```
mocha spec.js --require chai/register-assert.js # Using Assert style
```

```
mocha spec.js --require chai/register-expect.js # Using Expect style
```

```
mocha spec.js --require chai/register-should.js # Using Should style
```

Chai

Estos cadenas están disponibles para las afirmaciones de BDD (expect/should).

Alias Listado se puede usar indistintamente con su cadena original.

Encadenador	EJEMPLO
No	<code>.should('not.equal', 'Jane')</code> <code>expect(name).to.not.equal('Jane')</code>
profundo	<code>.should('deep.equal', { name: 'Jane' })</code> <code>expect(obj).to.deep.equal({ name: 'Jane' })</code>
anidado	<code>.should('have.nested.property', 'a.b[1]')</code> <code>.should('nested.include', {'a.b[1]': 'y'})</code> <code>expect({a: {b: 'x'}}).to.have.nested.property('a.b')</code> <code>expect({a: {b: 'x'}}).to.nested.include({'a.b': 'x'})</code>
ordenado	<code>.should('have.ordered.members', [1, 2])</code> <code>expect([1, 2]).to.have.ordered.members([1, 2])</code> <code>expect([1, 2]).not.to.have.ordered.members([2, 1])</code>
Cualquier	<code>.should('have.any.keys', 'age')</code> <code>expect(arr).to.have.any.keys('age')</code>
Todo	<code>.should('have.all.keys', 'name', 'age')</code> <code>expect(arr).to.have.all.keys('name', 'age')</code>

A(Tipo) Alias: Un	.should('be.a', 'string') expect('test').to.be.a('string')
Incluir(Valor) Alias: contener, incluye, contiene	.should('include', 2) expect([1,2,3]).to.include(2)
DE ACUERDO	.should('not.be.ok') expect(undefined).to.not.be.ok
verdadero	.should('be.true') expect(true).to.be.true
FALSO	.should('be.false') expect(false).to.be.false
nulo	.should('be.null') expect(null).to.be.null
indefinido	.should('be.undefined') expect(undefined).to.be.undefined
existir	.should('exist') expect(myVar).to.exist
vacío	.should('be.empty') expect([]).to.be.empty
Argumentos Alias: Argumentos	.should('be.arguments') expect(arguments).to.be.arguments
igual(Valor) Alias: iguales, eq	.should('equal', 42) expect(42).to.equal(42)
profundo(Valor)	.should('deep.equal', { name: 'Jane' }) expect({ name: 'Jane' }).to.deep.equal({ name: 'Jane' })

EQL(<i>Valor</i>) Alias: EQLS	.should('eq', { name: 'Jane' }) expect({ name: 'Jane' }).to.eq({ name: 'Jane' })
grande(<i>Valor</i>) Alias: GT, arriba	.should('be.greaterThan', 5) expect(10).to.be.greaterThan(5)
el menos(<i>Valor</i>) Alias: gte	.should('be.at.least', 10) expect(10).to.be.at.least(10)
menos(<i>Valor</i>) Alias: LT, abajo	.should('be.lessThan', 10) expect(5).to.be.lessThan(10)
mayoría(<i>Valor</i>) Alias: LTE	.should('have.length.of.at.most', 4) expect('test').to.have.length.of.at.most(4)
Dentro(<i>comenzar,finalizar</i>)	.should('be.within', 5, 10) expect(7).to.be.within(5, 10)
instancia de(<i>constructor</i>) Alias: instancia de	.should('be.instanceOf', Array) expect([1, 2, 3]).to.be.instanceOf(Array)
propiedad(<i>Nombre, [Valor]</i>)	.should('have.property', 'name') expect(obj).to.have.property('name')
profundo. Propiedad(<i>Nombre, [Valor]</i>)	.should('have.deep.property', 'tests[1]', 'e2e') expect(deepObj).to.have.deep.property('tests[1]', 'e2e')
propia propiedad(<i>Nombre</i>) Alias: HasOwnProperty, Own.Property	.should('have.ownProperty', 'length') expect('test').to.have.ownProperty('length')
OwnPropertyDescriptor(<i>Nombre</i>)	.should('have.ownPropertyDescriptor', 'a')

Alias: hanownpropertyDesc riptor	<code>expect({a: 1}).to.have.ownPropertyDescriptor('a')</code>
longitud (<i>Valor</i>)	<code>.should('have.lengthOf', 4)</code> <code>expect('test').to.have.lengthOf(4)</code>
fósforo (<i>Regexp</i>) Alias: partidos	<code>.should('to.match', /^test/)</code> <code>expect('testing').to.match(/^test/)</code>
cadena (<i>cadena</i>)	<code>.should('have.string', 'test')</code> <code>expect('testing').to.have.string('test')</code>
llaves (<i>llave1</i> , [<i>llave2</i>], [...]) Alias: llave	<code>.should('have.keys', 'pass', 'fail')</code> <code>expect({ pass: 1, fail: 2 }).to.have.keys('pass', 'fail')</code>
tirar (<i>constructor</i>) Alias: Lanza, lanza	<code>.should('throw', Error)</code> <code>expect(fn).to.throw(Error)</code>
responder (<i>método</i>) Alias: Respondersto	<code>.should('respondTo', 'getName')</code> <code>expect(obj).to.respondTo('getName')</code>
sí mismo	<code>.should('itself.respondTo', 'getName')</code> <code>expect(Foo).itself.to.respondTo('bar')</code>
satisfacer (<i>método</i>) Alias: satisfacer	<code>.should('satisfy', (num) => num > 0)</code> <code>expect(1).to.satisfy((num) => num > 0)</code>
closeto (<i>esperado, delta</i>) Alias: aproximadamente	<code>.should('be.closeTo', 1, 0.5)</code> <code>expect(1.5).to.be.closeTo(1, 0.5)</code>
Miembros (<i>colocar</i>)	<code>.should('include.members', [3, 2])</code> <code>expect([1, 2, 3]).to.include.members([3, 2])</code>
uno de (<i>valores</i>)	<code>.should('be.oneOf', [1, 2, 3])</code> <code>expect(2).to.be.oneOf([1,2,3])</code>

Cambiar(<i>función</i>) Alias: Cambios	.should('change', obj, 'val') expect(fn).to.change(obj, 'val')
aumentar(<i>función</i>) Alias: incremento	.should('increase', obj, 'val') expect(fn).to.increase(obj, 'val')
disminuir(<i>función</i>) Alias: disminución	.should('decrease', obj, 'val') expect(fn).to.decrease(obj, 'val')

Estos getters también están disponibles para las afirmaciones de BDD.
En realidad no lo hacen cualquier cosa, pero te permiten escribir oraciones claras en inglés.

Getters en cadena
to, be, been, is, that, which, and, has, have, with, at, of, same

chai-jQuery

Estos cadenas están disponibles al afirmar sobre un objeto DOM.

Usará comúnmente estas cadenas después de usar comandos DOM como: cy.get(), cy.contains(), etc..

Encadenadores	Afirmación
atencia(<i>Nombre</i>, [<i>Valor</i>])	.should('have.attr', 'bar') expect(\$el).to.have.attr('foo', 'bar')
apuntalar(<i>Nombre</i>, [<i>Valor</i>])	.should('have.prop', 'disabled', false) expect(\$el).to.have.prop('disabled', false)
CSS(<i>Nombre</i>, [<i>Valor</i>])	.should('have.css', 'background-color', 'rgb(0, 0, 0)')

	expect(\$el).to.have.css('background-color', 'rgb(0, 0, 0)')
Datos(<i>Nombre</i>, [<i>Valor</i>])	.should('have.data', 'foo', 'bar') expect(\$el).to.have.data('foo', 'bar')
clase(<i>nombre de clase</i>)	.should('have.class', 'foo') expect(\$el).to.have.class('foo')
IDENTIFICACIÓN(<i>IDENTIFICACIÓN</i>)	.should('have.id', 'foo') expect(\$el).to.have.id('foo')
HTML(<i>HTML</i>)	.should('have.html', 'I love testing') expect(\$el).to.have.html('with Cypress')
Texto(<i>Texto</i>)	.should('have.text', 'I love testing') expect(\$el).to.have.text('with Cypress')
Valor(<i>Valor</i>)	.should('have.value', 'test@dev.com') expect(\$el).to.have.value('test@dev.com')
visible	.should('be.visible') expect(\$el).to.be.visible
oculto	.should('be.hidden') expect(\$el).to.be.hidden
seleccionado	.should('be.selected') expect(\$option).not.to.be.selected
comprobado	.should('be.checked') expect(\$input).not.to.be.checked

Enfoque [Ed]	.should('have.focus') expect(\$input).not.to.be.focused expect(\$input).to.have.focus
activado	.should('be.enabled') expect(\$input).to.be.enabled
desactivado	.should('be.disabled') expect(\$input).to.be.disabled
vacío	.should('be.empty') expect(\$el).not.to.be.empty
existir	.should('exist') expect(\$nonexistent).not.to.exist
fósforo(<i>selector</i>)	.should('match', ':empty') expect(\$emptyEl).to.match(':empty')
contener(<i>Texto</i>)	.should('contain', 'text') expect(\$el).to.contain('text')
descendientes(<i>selector</i>)	.should('have.descendants', 'div') expect(\$el).to.have.descendants('div')

Sinon-chai

Estos cadenas se usan en afirmaciones con `cy.stub()` y `cy.spy()`.

SINON.JS PROPIEDAD/M ÉTODO	Afirmación
---	-------------------

Llamado	.should('have.been.called') expect(spy).to.be.called
personal de call	.should('have.callCount', 3) expect(spy).to.have.callCount(n)
Llamado	.should('have.been.calledOnce') expect(spy).to.be.calledOnce
Llamado	.should('have.been.calledTwice') expect(spy).to.be.calledTwice
Llamado THRICE	.should('have.been.calledThrice') expect(spy).to.be.calledThrice
llamado	.should('have.been.calledBefore', spy2) expect(spy1).to.be.calledBefore(spy2)
llamado	.should('have.been.calledAfter', spy2) expect(spy1).to.be.calledAfter(spy2)
llamado	.should('have.been.calledWithNew') expect(spy).to.be.calledWithNew
siempre llamado conNeo	.should('have.always.been.calledWithNew') expect(spy).to.always.be.calledWithNew
llamado	.should('have.been.calledOn', context) expect(spy).to.be.calledOn(context)
Alwayscalledon	.should('have.always.been.calledOn', context) expect(spy).to.always.be.calledOn(context)
llamado con	.should('have.been.calledWith', ...args) expect(spy).to.be.calledWith(...args)

siempre llamado con	.should('have.always.been.calledWith', ...args) expect(spy).to.always.be.calledWith(...args)
llamado	.should('have.been.calledOnceWith', ...args) expect(spy).to.be.calledOnceWith(...args)
Llamado Withitheactly	.should('have.been.calledWithExactly', ...args) expect(spy).to.be.calledWithExactly(...args)
siempre llamado con oxacta	.should('have.always.been.calledWithExactly', ...args) expect(spy).to.always.be.calledWithExactly(...args)
Llamado de la vez.	.should('have.been.calledOnceWithExactly', ...args) expect(spy).to.be.calledOnceWithExactly(...args)
llamadowithma tch	.should('have.been.calledWithMatch',...args) expect(spy).to.be.calledWithMatch(...args)
siempre llamado	.should('have.always.been.calledWithMatch',...args) expect(spy).to.always.be.calledWithMatch(...args)
devuelto	.should('have.returned', 'foo') expect(spy).to.have.returned(returnVal)
siempre retorcido	.should('have.always.returned', 'foo') expect(spy).to.have.always.returned(returnVal)
lanzar	.should('have.thrown', TypeError) expect(spy).to.have.thrown(errorObjOrErrorTypeS tringOrNothing)
siempre	.should('have.always.thrown', 'TypeError') expect(spy).to.have.always.thrown(errorObjOrErro rTypeStringOrNothing)

Mapa Conceptual calidad del software



Referencias bibliográficas

Assertion styles - Chai. (n.d.). Chaijs.com. Retrieved July 18, 2025, from <https://www.chaijs.com/guide/styles/>

Assertions. (n.d.). Cypress.io. Retrieved July 18, 2025, from <https://docs.cypress.io/app/references/assertions>

Bryan, D. (2024, January 24). *Understanding Chai: A comprehensive guide to the JavaScript assertion library.* Medium.
<https://medium.com/@dezsays/understanding-chai-a-comprehensive-guide-to-the-javascript-assertion-library-a382d5617832>